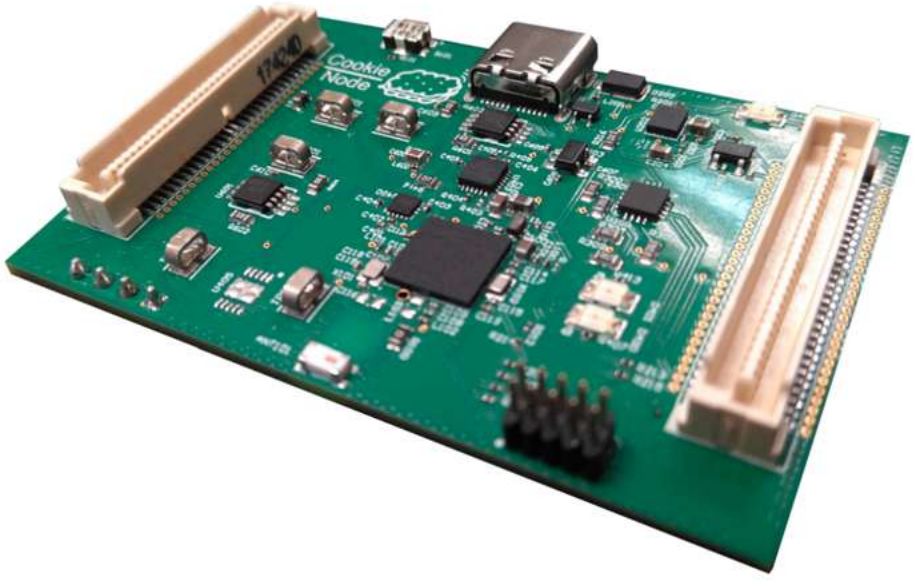
Escuela Técnica Superior de Ingenieros Industriales (UPM)

Wireless Sensor Networks

Lab 2 Report



Author: Jaime Pérez Pérez - 20259

Date: October 21, 2026

CONTENTS

1 Introduction **3**

2 Exercise 1 **4**

2.1 Software architecture 4

2.2 Sensor initialisation 4

2.3 Coordinator process 5

2.4 LEDs' and sensor processes 6

2.5 Results 6

3 Exercise 2 **8**

3.1 Information exchanged between processes 8

3.2 Command validation in the coordinator 8

3.3 Periodic execution with event timers 9

3.4 Results 11

APPENDICES **13**

A Code used in the first exercise 13

B Code used in the second exercise 15

1 Introduction

This second lab moves from the simulated environment used in Lab 1 (Cooja) to a real WSN hardware platform: the *Cookie Node*. This device is built around a Silicon Labs EFR32MG12 System on Chip (SoC) (ARM Cortex-M4). It integrates, among other elements, two LEDs (red and "green") and an SI7021 temperature and humidity sensor connected through an I^2C bus.

The firmware is developed using Contiki-NG, an Operating System (OS) used for IoT devices based on an event-driven architecture.
The solutions presented (and any other application developed with this OS) are structured in different processes that remain waiting until they "receive" an event addressed to them. These events include the expiration of a timer, the reception of a line through the serial port, or any user-defined events generated inside said processes.

This lab consists of two exercises that aim to use the serial terminal as a command line for the *Cookie Node*. The different actions the user can execute are:

- Toggle the *Cookie* LEDs. (**Exercise 1**).
- Request a measurement from the temperature or humidity sensor (**Exercise 1**).
- Configure the periodicity with which any of the previous actions is repeated (**Exercise 2**).

In order to execute said actions, the user must enter a specific frame through the command line. This way, the *Cookie* can identify which action is expected to be run, and inform the user if the action is unknown.
The developed frames are shown in Table 1.1.

The first character of each frame identifies the type of element (**L** for LEDs, **S** for any of the two sensors). The second one selects the specific LED (red or green) or sensor. In Exercise 2, an optional aditional number sets the period to repeat the indicated action in seconds.

Command	**Element**	**Action**	**Period (Ex. 2)**
`LR[N]`	Red LED	Toggle the LED	Every N s
`LG[N]`	Green LED	Toggle the LED	Every N s
`ST[N]`	SI7021 sensor	Measure temperature (^{o}C)	Every N s
`SH[N]`	SI7021 sensor	Measure humidity (%)	Every N s

Table 1.1: Frame format of the commands sent through the serial terminal.

The first exercise only accepts the first two characters, while in exercise 2, N omitted or $N = 0$ means a single execution of the introduced command.

The full source code of both exercises can be found in this document's appendices.

2 Exercise 1

The first exercise expects a message through the command line to trigger actions on demand: the node stays idle until a command arrives, executes it once and goes back to waiting.

2.1 Software architecture

The application has been split into four processes (see Table 2.1), all of them started automatically thanks to Contiki's `AUTOSTART_PROCESSES` function.

Process	Responsibility
`sensors_setup`	Configures and starts the temperature/humidity sensor & initialises the I^2C bus. It runs only once.
`coordinator`	Waits for serial line events, decodes the command and triggers the right process.
`toggle_leds`	Toggles the red or green LED.
`read_sensors`	Performs a measurement and prints the requested magnitude (temperature or humidity).

Table 2.1: Processes used in Exercise 1.

2.2 Sensor initialisation

Before any measurement can be taken, the sensor block of the Cookie must be enabled. Pin `PF9` handles the analog switch that powers the temperature/humidity circuit, so it is configured as a push-pull (output) and set high.

Afterwards, the I^2C is initialised with its default configuration and the sensor is started.

The code in charge of this setup is the following:

```
1 GPIO_PinModeSet(gpioPortF, 9, gpioModePushPull, 1); // T/H power switch as
          ↪ output
2 GPIO_PinOutSet(gpioPortF, 9);                       // T/H circuit enabled
3
4 I2CSPM_Init_TypeDef i2cspmInit = I2CSPM_INIT_DEFAULT;
5 I2CSPM_Init(&i2cspmInit);                           // I2C bus configuration
6 SI7021_init();                                      // Sensor configuration
7
```

Code 2.1: Sensor setup process.

2.3 Coordinator process

The coordinator is the entry point of every command. It allocates a user defined event (`coordinator_event`) using the `process_alloc_event()` function. This way, the LEDs and Sensor can wait until receiving said coordinator event.

The process starts by waiting until a command is received through the serial line. Whenever this happens, the coordinator event is triggered, receiving the introduced data as a string.

The data received is compared with the 4 known frames: ("LR", "LG", "ST" & "SH"). If it matches any of those, then the coordinator triggers the right process to take care of the desired action.

- To toggle any of the LEDS, the `toggle_leds` process is launched, receiving "RED" or "GREEN" inside the data string, indicating specifically which LED has to toggle.
- If the temperature or humidity is expected, the `read_sensors` process must be run. Following the same procedure, this process receives "H" or "T" depending on the case.

If the frame received doesn't match any of these four, then the user is informed through the serial monitor, indicating the valid messages.

```
1  PROCESS_WAIT_EVENT_UNTIL(ev==serial_line_event_message);
2  printf("\nMessage received: [%s]",(char*)data);
3
4  if(strcmp(data,"LR") == 0) process_post(&toggle_leds,coordinator_event,"RED");
5
6  else if(strcmp(data,"LG") == 0)
           ↪ process_post(&toggle_leds,coordinator_event,"GREEN");
7
8  else if(strcmp(data,"SH") == 0)
           ↪ process_post(&read_sensors,coordinator_event,"H");
9
10 else if(strcmp(data,"ST") == 0)
           ↪ process_post(&read_sensors,coordinator_event,"T");
11
12 else{
13     printf(" not identified. Valid messages are: LR, LG, SH, ST\n");
14     printf("\nWaiting for next serial line reception\n");
15 }
16
```

Code 2.2: Coordinator process. (Exercise 1)

As seen in Code 2.2, the comparison between the "data" and each known frame is done using C's string comparison function `strcmp()`.

2.4 LEDs' and sensor processes

Both `toggle_leds` and `read_sensors` follow the same pattern: they wait until receiving the *coordinator event* (`PROCESS_WAIT_EVENT_UNTIL(ev == coordinator_event)`). In the meantime, they give the control to the scheduler, following Contiki's functioning method.

When the event is received, each process decodes the received data. In the `toggle_leds` process, the right LED is toggled using the `leds_toggle()` function. This action is also informed through the serial monitor, in order to log the results.

In case of the sensors, whenever the *coordinator event* is received, then the `SI7021_measure()` function performs the measurements, saving them in the right variables. Then they are processed to show the right units, and displayed through the serial monitor so that the user can read the values.

2.5 Results

Figures 2.1 to 2.3 show the behaviour of the node when the different commands are introduced through CoolTerm.

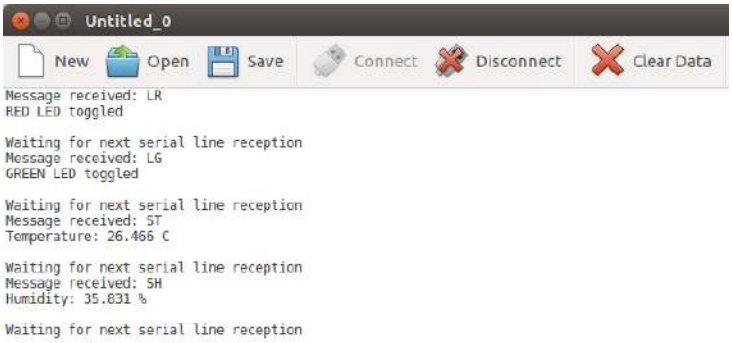
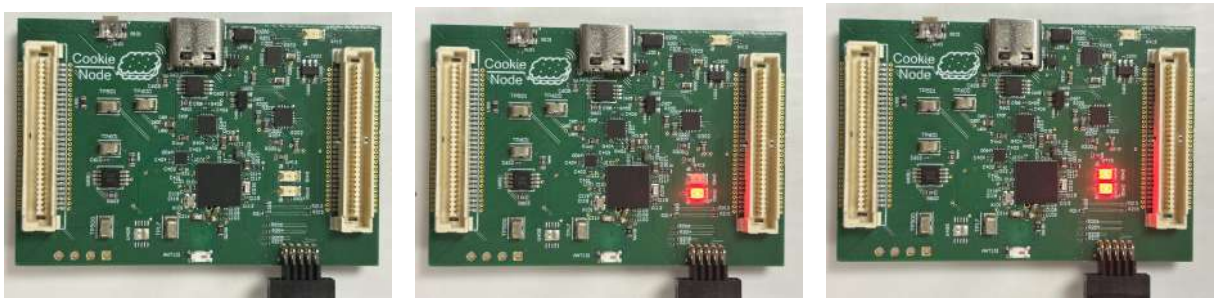


Figure 2.1: Serial terminal output after sending `ST`, `SH`, `LR` and `LG`.



(a) Both LEDs OFF (b) After `LR` (c) After `LG`

Figure 2.2: State of the Cookie LEDs after each LED command.

Figure 2.2 shows the LEDs behavior: each `LR` or `LG` command inverts the state of the selected LED. Receiving the same command again returns each LED to its previous state (OFF).

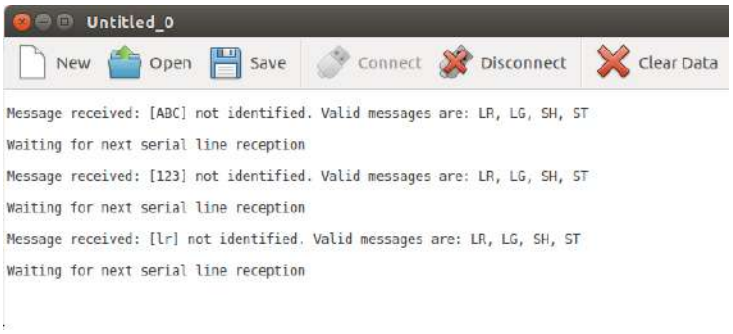


Figure 2.3: Response of the node to an unknown command.

Finally, Figure 2.3 shows that unrecognised inputs are discarded without triggering any process, and the user is informed of the valid options.

3 Exercise 2

The second exercise is an extension of the previous one. Now, the user can also configure how often each action is executed.

To do so, at the end of each valid frame, a number can be introduced. This value sets the period in seconds. For example, `ST2` measures and displays the temperature every 2 seconds.
Omitting the final number, or setting it to 0, keeps the single-execution behaviour of Exercise 1. Each of the four actions can have its own period, and all of them can run simultaneously.

The process architecture is the same as previously seen. The main differences are the use of timers and the fact that the coordinator now decodes the command line messages and forwards them to the right process.

3.1 Information exchanged between processes

In Exercise 1 a simple string was enough to describe the action. Now each command includes two pieces of information: the action to be performed, and the periodicity to do so. To handle both, a struct structure has been defined and used throughout this application:

```
1 struct command{
2     char action; // 'R', 'G', 'H' or 'T'
3     int period;  // Period in seconds (0 = single execution)
4 };
5
```

Code 3.1: Structure sent from the coordinator to the other processes.

It's important to highlight that only the second character of each command is stored in the `action` variable within the struct. This is because the first one is implicit in the process to be triggered. Receiving an 'L' means launching `toggle_leds`, while a 'S' triggers `read_sensors`.

3.2 Command validation in the coordinator

Because the frame now has a variable length, the coordinator validates every incoming line in three consecutive steps before forwarding it. If any of them fails, an error message is printed and the loop jumps back (`continue`) to wait for the next line:

1. **Length**: the message must contain at least two characters.
2. **Content**: the first two characters must form one of the valid pairs (`LR`, `LG`, `ST`, `SH`). If this condition is met, then the destination process is selected accordingly.
3. **Period**: the rest of the string is converted with C's function `strtol()`:

   `strtol()` receives the string to analyze, a pointer indicating where it stopped reading, and the base in which the strings contain the data (in this case, 10 represents decimal). This function is in charge of reading only numbers.

The `end` pointer returned by the function points to where the function has stopped. If the reading has been successful, then it must point to the null terminator character contained by any string ("\0"). If the element read is not a number, then `strtol` stops and points to said element.

(For example, a `ST2f` command will return 'f' in the `end` pointer's value).

This value is then checked to verify the `strtol` result, informing the user if the reading was invalid.

The valid periods in this exercise have been reduced from 0 to 30. Any other period that fits into a *integer* could work without problems, but in order to simplify the results shown in the serial monitor, the `MAX_PERIOD` has been set to 30.

```
user_period = strtol(&incoming_data[2], &end, 10);
if((*end != '\0') || ((int)user_period<0) ||((int)user_period>MAX_PERIOD) ){
    printf(" invalid period. Format expected is a number from 0 to %d\n",
        ↪ MAX_PERIOD);
    continue;
}
data_to_next_process.action = incoming_data[1];
data_to_next_process.period = (int)user_period;

process_post(next_proc, coordinator_event, &data_to_next_process);

```

Code 3.2: Validation of the period field.

Once the command is checked to be valid, it is forwarded using `process_post`.

- The destination (`next_proc`) is stored when reading a 'L' or 'S' in the first character of the command line message.
- The data forwarded is stored using the struct previously seen. In order to send it correctly, it's sent as a pointer. This way, the next process will decode the content and extract the information it needs.

  Since the structure is stored as a `static` variable, its content is never overwritten before finishing the coordinator's process.

3.3 Periodic execution with event timers

Periodicity is implemented with Contiki-NG *etimers*. Each action has its own timer (`red_timer`, `green_timer`, `temp_timer` and `humidity_timer`), declared as `static` so that they keep their value after each process gives control back to the scheduler.

Having one timer per action allows to run them simultaneously and in the user's desired periods.

To achieve this behavior, now each process (both the LEDs' and the sensors') waits for either one of the two possible events:

- `Coordinator_event`: This means that a new command has been received. The wanted action is run automatically. Then, if the period received is greater than 0, the corresponding timer is reset with this new count value (using `etimer_set()`).

In previous versions, `etimer_reset_with_new_interval()` was used to change the timer dinamically. However, this function uses the previous timers' expiration timestamp. Whenever this was used, the timer expired repeatedly until it caught up with the current time. Using `etimer_set()` saves the period correctly, avoiding multiple expirations before catching up.

If the received period is 0 or doesn't exist (none introduced), then the timer is stopped (with `etimer_stop()`).

- `PROCESS_EVENT_TIMER`: This means that one of the timers has expired. The first thing to do is checking what timer specifically launched this event, and then the associated action is executed and the timer restarted (with `etimer_reset()`)

The `leds_toggle` process is shown as an example (Code 3.3). The sensor's process follows the same structure.

```
if(ev==coordinator_event){

    //New data received from coordinator.
    incoming_data = (struct command*)data;
    printf("\nAction received: %c. Period: %d\n",
          ↪ incoming_data->action,incoming_data->period);

    if(incoming_data->action == 'R'){
        leds_toggle(LEDS_RED);
        if(incoming_data->period >0){
            etimer_set(&red_timer,
            (clock_time_t)incoming_data->period * CLOCK_SECOND);
        }
        else etimer_stop(&red_timer);
    }

    else if(incoming_data->action=='G'){
    //(... Same as the red LED)
    }
}
else if(ev==PROCESS_EVENT_TIMER){
    if(data==&red_timer){
        leds_toggle(LEDS_RED);
        etimer_reset(&red_timer);
        printf("(Timestamp: %d) Red LED toggled", (int)(clock_time()/1000));
    }
    else if(data==&green_timer){
        leds_toggle(LEDS_GREEN);
        etimer_reset(&green_timer);
        printf("(Timestamp: %d) Green LED toggled", (int)(clock_time()/1000));
    }

}

```

Code 3.3: Handling of the two possible events in the `toggle_leds` process.

In the sensor process, the periodic messages also print the system time (`clock_time()`, in seconds), which makes it possible to check in the terminal that the measurements are actually separated by the configured period.

3.4 Results

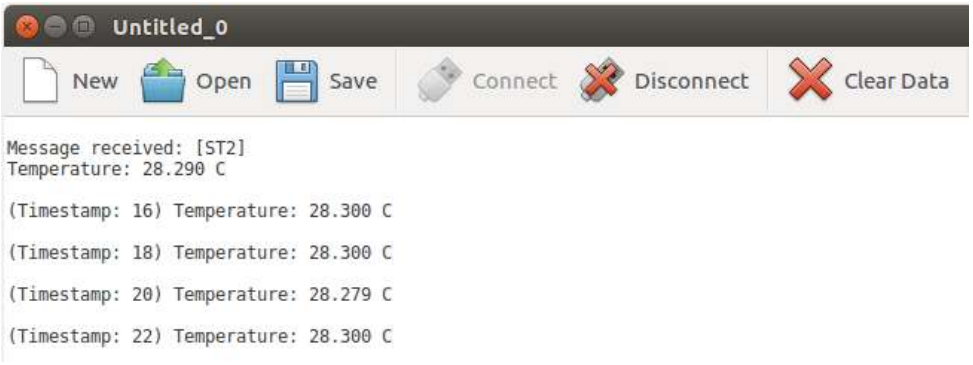


Figure 3.1: Periodic temperature measurements after sending `ST2`.

Figure 3.1 shows the output after sending `ST2`. The first measurement is printed immediately and, from then on, a new one is reported periodically. The difference between consecutive timestamps shows the 2 seconds between readings.

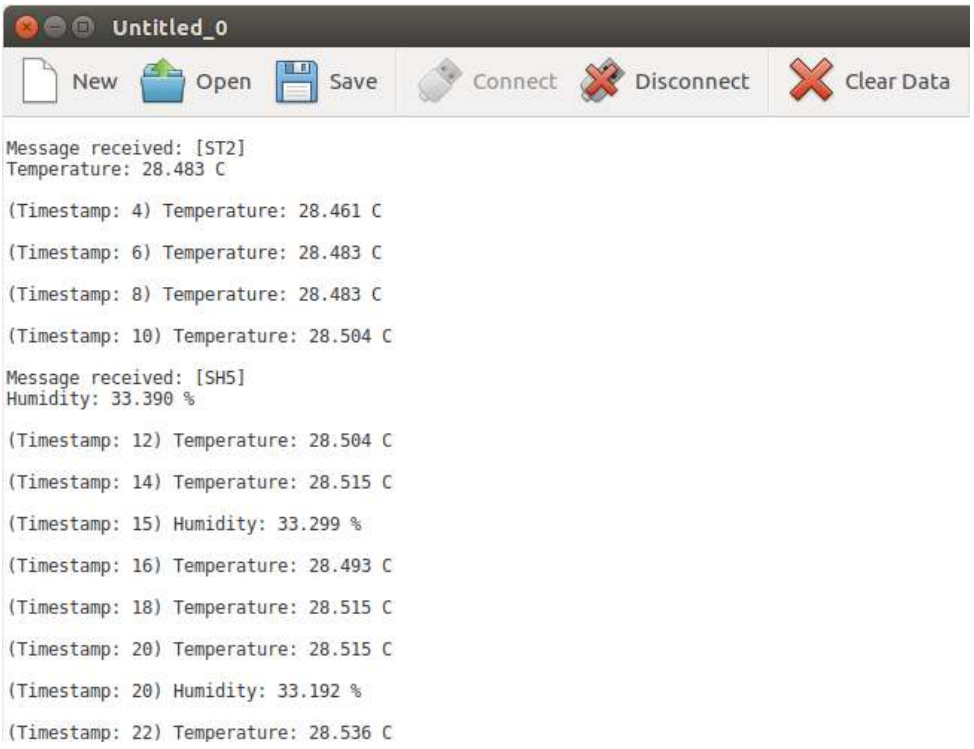


Figure 3.2: Temperature (`ST2`) and humidity (`SH5`) reported simultaneously with different periods.

As seen in Figure 3.2, several periodic actions can coexist, each one with its own period, since they are driven by independent timers.

Logically, this can be done with the LEDs as well, as seen in Figure 3.3

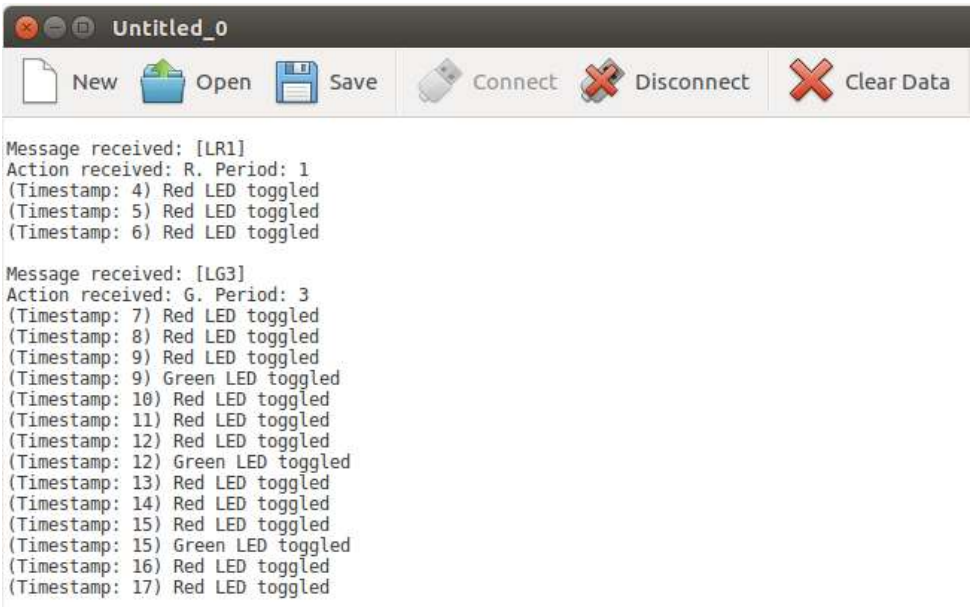


Figure 3.3: LEDs blinking at different periods thanks to their own timers.

The periodic behavior, as stated in this second exercise, can be stopped by sending the same command with period 0 or without an explicit period. Figure 3.4 shows exactly that: the temperature sensor periodicity is stopped by sending a ST command.



Figure 3.4: Stopping a periodic action with `ST`.

Finally, Figure 3.5 shows the response of the coordinator to the three types of error considered: message too short (`S`), unknown (`XY3`) and invalid period (`ST2a`).

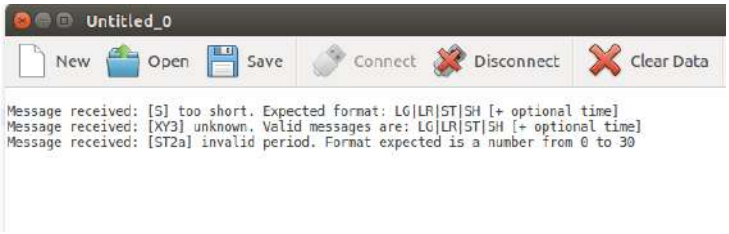


Figure 3.5: Error messages shown by the coordinator.

APPENDICES

A Code used in the first exercise

```
#include "contiki.h"
#include <stdio.h> /* For printf() */
#include <stdlib.h>
#include "leds.h"
#include "serial-line.h"
#include "string.h"
#include "efr32mg12p432f1024gl125.h"
#include "em_cmu.h"
#include "em_emu.h"
#include "si7021.h"
#include "board_cookie.h"
#include "util.h"
#include "imu.h"
#include "icm20648.h"

process_event_t coordinator_event;

PROCESS(sensors_setup, "Sensors setup");
PROCESS(coordinator, "Coordinator process");
PROCESS(toggle_leds, "Toggle LEDs process");
PROCESS(read_sensors, "Sensors reading");

AUTOSTART_PROCESSES(&sensors_setup,&coordinator,&toggle_leds,&read_sensors);


PROCESS_THREAD(sensors_setup, ev ,data)
{
    PROCESS_BEGIN();

    GPIO_PinModeSet(gpioPortF, 9, gpioModePushPull, 1); // T/H, set as out pin
        ↪ (PushPull)
    GPIO_PinOutSet(gpioPortF, 9); // T/H, enabled

    I2CSPM_Init_TypeDef i2cspmInit = I2CSPM_INIT_DEFAULT; // I2C configuration
    I2CSPM_Init(&i2cspmInit); // I2C configuration
    SI7021_init(); // TEMP/HUM Sensor config

    PROCESS_END();
}

PROCESS_THREAD(coordinator,ev, data){

    PROCESS_BEGIN();

    coordinator_event = process_alloc_event();
    while(1){

        PROCESS_WAIT_EVENT_UNTIL(ev==serial_line_event_message);
        printf("\nMessage received: [%s]",(char*)data);

        if(strcmp(data,"LR") == 0)
          ↪ process_post(&toggle_leds,coordinator_event,"RED");

```

```
52
53          else if(strcmp(data,"LG") == 0)
              ↪ process_post(&toggle_leds,coordinator_event,"GREEN");
54
55          else if(strcmp(data,"SH") == 0
              ↪ )process_post(&read_sensors,coordinator_event,"H");
56
57          else if(strcmp(data,"ST") == 0)
              ↪ process_post(&read_sensors,coordinator_event,"T");
58
59          else{
60            printf(" not identified. Valid messages are: LR, LG, SH, ST\n");
61            printf("\nWaiting for next serial line reception\n");
62          }
63
64      }
65      PROCESS_END();
66  }
67
68  PROCESS_THREAD(toggle_leds,ev, data){
69
70      PROCESS_BEGIN();
71
72      while(1){
73
74          PROCESS_WAIT_EVENT_UNTIL(ev == coordinator_event);
75          printf("\n%s LED toggled\n",(char*)data);
76          printf("\nWaiting for next serial line reception\n");
77          if(strcmp(data,"RED")==0) leds_toggle(LEDS_RED);
78          else if(strcmp(data,"GREEN")==0) leds_toggle(LEDS_GREEN);
79
80      }
81      PROCESS_END();
82  }
83
84  PROCESS_THREAD(read_sensors,ev, data){
85
86      uint32_t hume = 0;
87      int32_t temper = 0;
88
89      PROCESS_BEGIN();
90
91      while(1){
92
93          PROCESS_WAIT_EVENT_UNTIL(ev == coordinator_event);
94
95          SI7021_measure(&hume, &temper);
96
97          if((strcmp(data,"H"))==0) printf("\nHumidity: %d.%d %%\n",
              ↪ (int)(hume/1000), (int)(hume%1000));
98
99          else if((strcmp(data,"T"))==0) printf("\nTemperature: %d.%d C\n",
              ↪ (int)(temper/1000), (int)(temper%1000));
100         printf("\nWaiting for next serial line reception\n");
101
102     }
103     PROCESS_END();
104 }
105
```

Code A.4: Exercise 1 full code.

B Code used in the second exercise

```c
#include "contiki.h"
#include <stdio.h> /* For printf() */
#include <stdlib.h>
#include "leds.h"
#include "serial-line.h"
#include "string.h"
#include "efr32mg12p432f1024gl125.h"
#include "em_cmu.h"
#include "em_emu.h"
#include "si7021.h"
#include "board_cookie.h"
#include "util.h"
#include "imu.h"
#include "icm20648.h"
#include "etimer.h"

#define MAX_PERIOD 30
process_event_t coordinator_event;

struct command{
    char action; // Will contain R,G,H or T. This way, I can use one single process to toggle LEDs and other to read sensors.
    int period;  // If the user selects a period to perform said action.
};

PROCESS(sensors_setup, "Sensors setup");
PROCESS(coordinator, "Coordinator process");
PROCESS(toggle_leds, "Toggle LEDs process");
PROCESS(read_sensors, "Sensors reading");

AUTOSTART_PROCESSES(&sensors_setup,&coordinator,&toggle_leds,&read_sensors);

PROCESS_THREAD(sensors_setup, ev ,data)
{
    PROCESS_BEGIN();

    GPIO_PinModeSet(gpioPortF, 9, gpioModePushPull, 1); // T/H, set as out pin (PushPull)
    GPIO_PinOutSet(gpioPortF, 9); // T/H, enabled

    I2CSPM_Init_TypeDef i2cspmInit = I2CSPM_INIT_DEFAULT; // I2C configuration
    I2CSPM_Init(&i2cspmInit); // I2C configuration
    SI7021_init(); // TEMP/HUM Sensor config

    PROCESS_END();
}

```

```
45 PROCESS_THREAD(coordinator,ev, data){
46
47     static struct command data_to_next_process;
48     struct process *next_proc;
49     long user_period;
50     char *end;
51     char *incoming_data;
52
53     PROCESS_BEGIN();
54
55     coordinator_event = process_alloc_event();
56
57     while(1){
58
59         PROCESS_WAIT_EVENT_UNTIL(ev==serial_line_event_message);
60
61         incoming_data = (char*)data;
62
63         printf("\nMessage received: [%s]",(char*)data);
64
65         //First, I check if the message is valid: It must have 2 characters and
             ↪ an optional number:
66         if(strlen(data)<2){
67             printf(" too short. Expected format: LG|LR|ST|SH [+ optional
             ↪ time]");
68             continue;
69         }
70
71         //If the length is right, now I check if the content is valid (LR,LR,ST
             ↪ or SH)
72         if(incoming_data[0] == 'L' && (incoming_data[1]=='R'||
             ↪ incoming_data[1]=='G')) next_proc = &toggle_leds;
73
74         else if(incoming_data[0] == 'S'
             ↪ &&(incoming_data[1]=='H'||incoming_data[1]=='T')) next_proc =
             ↪ &read_sensors;
75
76         else{
77             printf(" unknown. Valid messages are: LG|LR|ST|SH [+ optional
             ↪ time]");
78             continue; //To stop running this process.
79         }
80
81
82         //The length and content are valid so far. Now I check if the received
             ↪ time is valid:
83         user_period = strtol(&incoming_data[2],&end,10);
84         if((*end != '\0') || ((int)user_period<0)
             ↪ ||((int)user_period>MAX_PERIOD) ){
85             printf(" invalid period. Format expected is a number from 0 to
             ↪ %d\n",MAX_PERIOD);
86             continue;
87         }
88
89         //If the code reaches here, the message is totally valid. Now I unpack
             ↪ it and send it to the right next process
90         data_to_next_process.action = incoming_data[1];
91         data_to_next_process.period = (int)user_period;
92
93         process_post(next_proc,coordinator_event,&data_to_next_process);
94
95     }
96     PROCESS_END();
97 }
98
```

```
99  PROCESS_THREAD(toggle_leds,ev, data){
100
101     struct command *incoming_data;
102     static struct etimer red_timer, green_timer;
103
104     PROCESS_BEGIN();
105
106     while(1){
107         PROCESS_WAIT_EVENT_UNTIL(ev==coordinator_event ||
108         ev==PROCESS_EVENT_TIMER);
109
110
111             if(ev==coordinator_event){
112                 //New data received from coordinator. I check the target LED
          ↪ and its periodicity
113                 incoming_data = (struct command*)data;
114                 printf("\nAction received: %c. Period:
          ↪ %d\n",incoming_data->action,incoming_data->period);
115
116                 if(incoming_data->action == 'R'){
117                     leds_toggle(LEDS_RED);
118                     if(incoming_data->period
          ↪ >0)etimer_set(&red_timer,(clock_time_t)incoming_data->period *
          ↪ CLOCK_SECOND);
119
120                     else etimer_stop(&red_timer);
121                 }
122
123                 else if(incoming_data->action=='G'){
124                     leds_toggle(LEDS_GREEN);
125                     if(incoming_data->period
          ↪ >0)etimer_set(&green_timer,(clock_time_t)incoming_data->period *
          ↪ CLOCK_SECOND);
126                     else etimer_stop(&green_timer);
127                 }
128             }
129
130             else if(ev==PROCESS_EVENT_TIMER){
131                 if(data==&red_timer){
132                     leds_toggle(LEDS_RED);
133                     etimer_reset(&red_timer);
134                     printf("(Timestamp: %d) Red LED toggled\n",
          ↪ (int)(clock_time()/1000));
135                 }
136                 else if(data==&green_timer){
137                     leds_toggle(LEDS_GREEN);
138                     etimer_reset(&green_timer);
139                     printf("(Timestamp: %d) Green LED toggled\n",
          ↪ (int)(clock_time()/1000));
140                 }
141             }
142
143
144     }
145     PROCESS_END();
146 }
147
```

```
PROCESS_THREAD(read_sensors,ev, data){

    static uint32_t hume = 0;
    static int32_t temper = 0;

    struct command *incoming_data;
    static struct etimer temp_timer, humidity_timer;

    PROCESS_BEGIN();

    while(1){

        PROCESS_WAIT_EVENT_UNTIL(ev ==
          ↪ coordinator_event||ev==PROCESS_EVENT_TIMER);

        if(ev==coordinator_event){
            incoming_data = (struct command*)data;
            SI7021_measure(&hume, &temper);

            if(incoming_data->action=='H'){
                printf("\nHumidity: %d.%d %%\n", (int)(hume/1000),
          ↪ (int)(hume%1000));
                if(incoming_data->period > 0
          ↪ )etimer_set(&humidity_timer,(clock_time_t)incoming_data->period *
          ↪ CLOCK_SECOND);
                else etimer_stop(&humidity_timer);
            }
            else if(incoming_data->action=='T'){
                printf("\nTemperature: %d.%d C\n", (int)(temper/1000),
          ↪ (int)(temper%1000));

                if(incoming_data->period > 0
          ↪ )etimer_set(&temp_timer,(clock_time_t)incoming_data->period *
          ↪ CLOCK_SECOND);
                else etimer_stop(&temp_timer);
            }
        }

        else if(ev==PROCESS_EVENT_TIMER){
            SI7021_measure(&hume, &temper);

            if(data==&temp_timer){
                printf("\n(Timestamp: %d) Temperature: %d.%d
          ↪ C\n",(int)(clock_time()/1000), (int)(temper/1000),
          ↪ (int)(temper%1000));
                etimer_reset(&temp_timer);
            }

            else if(data==&humidity_timer){
                printf("\n(Timestamp: %d) Humidity: %d.%d %%\n",
          ↪ (int)(clock_time()/1000),(int)(hume/1000), (int)(hume%1000));

                etimer_reset(&humidity_timer);
            }
        }


    }
    PROCESS_END();
}

```

Code B.5: Exercise 2 Full code.